University of Southern California

Cardinal Processor
Project Phase 2

Objective:
The goal is for students to design and verify the 64-bit variable data width Cardinal processor using an RTL style of Verilog. The emphasis is on correct functionality, not timing, for this part of the project. You should not be using delays in the design itself, as we will attempt to synthesize the code in later stages of the project. To establish a point of consistency across all projects, simulate at a clock cycle of 4ns (250 MHz). For further verification of your processor's functionality, you will also write a test bench to test the processor architecture. Except for the instruction and data memory, all of your design should be fully synthesizable to be prepared for Part 4 of the project.

Team Policies:
Everyone should continue to work in their 2-student teams for the project unless already granted permission to work individually on the project this semester.

Verilog Setup:
Use the Verilog environment that you used for Project Part-1 to complete this project phase. Specifically, use the NC-Sim environment that has been recommended in the class. Codes compiling with other tools but not with the NC-Sim environment will be penalized severely, in a similar manner to a non-compiling code.

What to do?

- Download the Cardinal Processor Instruction Set Architecture (ISA) Manual from the course website.
- Download the provided tar file, which provides a framework for simulations that uses a makefile to compile and test your design. The provided tar file includes Instruction/Data memory code (imem.v and dmem.v) and the reference top-level test-bench design (tb_cmp.v, for reference only in this part). You are required to strictly follow the port names as specified in the top-level design. You are required to submit your designs in the provided directory structure, as the TAs/mentor will use the makefile to compile and test your code. You are free to use your own framework for designing and testing sub-modules of your processor code.
- Write RTL Verilog code for a variable-width 4-stage pipelined processor that executes all the instructions in the Cardinal Processor Instruction Set Manual. (Since Load/Store instructions use only immediate address specifiers, memory operations can be performed in the same stage as ALU functions; therefore, a 4-stage pipeline results.) The pipeline stages are then –
    - Instruction Fetch (IF)
    - Instruction Decode and Register Fetch (ID)
    - Execution or Memory Access (EX/MEM)
    - Write Back (WB)

  A general high-level diagram is shown in Figure 1. The top-level module has only clock and reset inputs and memory ports for the instruction and data memories. You may need to add control multiplexers or other building blocks in addition to these. All pipeline stage registers are assumed to be clocked on the rising edge of the clock. You may also wish to connect the buses slightly differently. For instance, it is possible to feed certain bits of the instructions directly to ports of the register file without going through the Instruction Decode block. Furthermore, few of the control connections are shown, such as ALU control, data memory control, etc. A short description of the elements shown in Figure 1 follows:

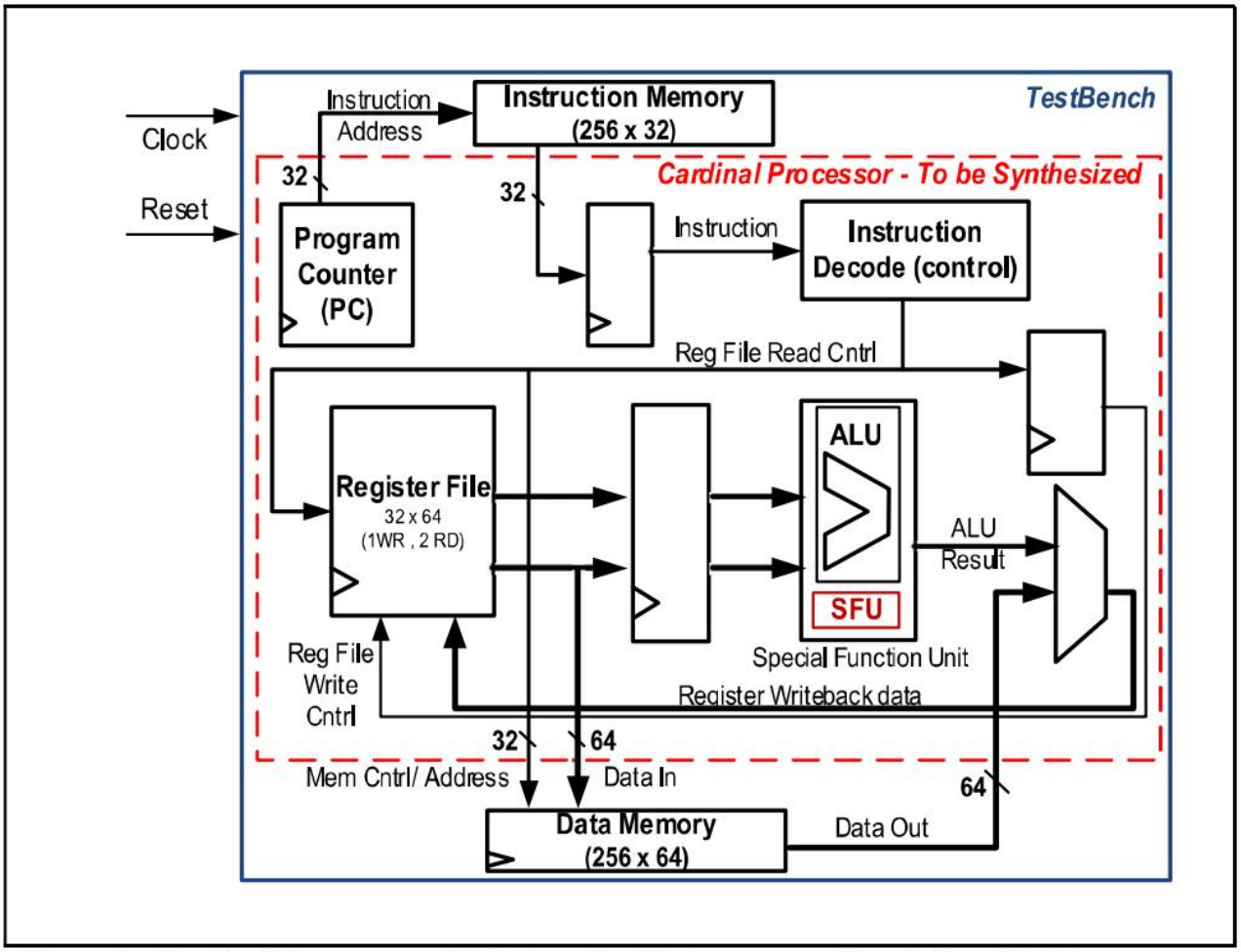

Figure 1: Cardinal Processor with Instruction/Data memory Simulation Models

- The system assumes a SYNCHRONOUS active-high RESET. At reset, you should clear all the contents of stage registers.
- The Program Counter (PC) unit should implement a 32-bit PC which increments by 4 every clock and has a value of 32'h0000_0000 at reset. The PC should be positive-edge triggered with a synchronous reset.
- An asynchronous instruction memory which is 256 words by 32 bits/word should be sufficient for the testbench programs in the project, and it can be assumed to support READ operations ONLY. This component is part of your testbench and is not required to be synthesized. This is provided to you and should not be modified.
- The Instruction Decode unit should decode the fetched instruction and generate all necessary control signals for the correct execution of the fetched instruction.
- The Register File is a 3-ported (2-READ port and 1-WRITE port) general-purpose register file that implements 32 64-bit registers. The reading from the register file is asynchronous, while writing to the register file is synchronous. Each port consists of a 5-bit address (address specifier) and 64-bit data for reading/writing the contents to the specified register. At reset, all register contents must be cleared. Register 0 needs to be hard-wired to 64'h0000_0000_0000_0000 and is READ ONLY, i.e., register 0 cannot be written to. The write port should also contain a write enable signal (wrEn) to control write operations. The read ports do not have enable signals, so each read port

will read the register contents based on the associated 5-bit address presented (asynchronously).

- The ALU block performs all the operations specified in the Instruction Set Architecture (ISA) manual instructions.
- The SFU (Special Function Unit) block performs higher-level operations. You are required to instantiate Synopsys DesignWare components for implementing higher mathematical functions such as square root and division.
- A synchronous data memory that is 256 words by 64 bits/word would be sufficient for the testbench programs in the project. Unlike the Instruction Memory, the Data Memory must support both read and write operations. This component is part of your testbench and is not required to be synthesized. The data memory is provided to you and should not be modified.
- You are required to implement two different branch instructions. The branch instruction requires reading a register-file value and deciding whether the branch is to be executed. As one can read the contents of an entry in the register file in the Instruction Decode (ID) stage, you must compare the register-file values and decide whether the branch is to be taken in the same stage (ID stage). In case the branch is taken, you should update the PC with the immediate address value (branch target). The new instruction will be fetched in the next clock cycle. In this case, you need to flush one instruction behind the branch instruction or turn it into a bubble — that is, the instruction in the Instruction Fetch (IF) stage. This processor will not execute delayed-slot branches. Flushing an instruction means the instruction should not update any stage registers like Instruction/Data Memory or register files. If the branch instruction is not taken, the next instruction in the IF stage should be decoded and executed normally.
- When using Synopsys DesignWare components, various combinational and sequential models are available. Select the one that best suits your processor requirements.

- Build your own testbench to verify your design, using the provided testbench as a reference. The good thing about processor designs is that they can be verified by assembly programs. The provided testbench loads your program into your Instruction Memory from file "imem.fill" and initial data needed into the Data Memory from file "dmem.fill." You may set the cycle time of your design by modifying "CYCLE_TIME" defined at the top of your testbench. A sample test assembly program is provided. You should write more elaborate test benches to exercise every instruction in the instruction set at least once, as well as every possible setting of WW bits (refer to the instruction manual). Make sure when testing different WW settings, you use appropriate instructions and data settings to verify that operand width selection works properly. Your code should cause unique values to be written into registers to verify that each instruction execution had the desired effect.
- Compile and test your processor. You may wish to perform pre-tests on subcomponent modules before integrating all the modules.

<u>What to Submit?</u>

Submit in the directory structure specified for your processor design, including the complete Verilog code, the testbench, and testcases used for your verification. Please include adequate comments not only in your design but also in your testbench to justify that your testbench adequately verifies your design.

In addition, include the program and data files that are loaded into the Instruction Memory and Data Memory for testing purposes. Please comment your binary assembly code with the corresponding assembly mnemonics (in .fill files).

How to Submit?

"Tar" the directory which contains all the requested files, and use the following commands for electronic submission:

- cd to the directory which contains all the files requested and only the files requested.
- "Tar" the directory (the tar filename should include the names of both project partners):
  FirstName1_LastName1_FirstName2_LastName2
  Do NOT include other files such as waveform files and log files.
  > *tar cvf tar_filename.tar .* (← Don't forget "the dot" at the end.)